

Aero Arc Telemetry Normalization Plan

Relay normalization design and InfluxDB workstream continuation

July 10, 2026

Status: proposed implementation plan

Executive Summary

Aero Arc can currently capture and forward approximately 226 messages from the gomavlib common dialect. The agent uses generated code to extract MAVLink fields, writes frames to its WAL, and sends a generic TelemetryFrame to the relay. The relay wraps that frame in a TelemetryEnvelope and routes it to configured outputs.

This provides broad forwarding, but it is not semantic normalization. Field values cross the gRPC boundary as strings, retain MAVLink names and scaling, and do not yet form a stable query contract. The current InfluxDB sink is scaffold-level and does not create a usable point.

The next step is to preserve the generic forwarding path while adding an Aero Arc-owned normalization path for internal telemetry. Selected messages will be parsed into typed values, converted into documented units, assigned stable field names, and written to InfluxDB through a dedicated telemetry backend.

This work directly continues the platform workstreams document. It advances implementation order item 1, "Lock identity and telemetry schema," and item 2, "Finish relay InfluxDB sink mapping." It is the relay half of the recommended production vertical slice:

Agent WAL -> Relay -> InfluxDB -> API latest telemetry

Current State

Agent extraction

The agent receives typed gomavlib messages. A generation-time reflection tool enumerates common.Dialect.Messages and emits a type-safe commonFields switch. The generated switch currently covers approximately 226 message types and avoids runtime reflection in the hot path.

The extracted values are stored in TelemetryFrame.Fields as map<string, string>. This means a numeric MAVLink value such as latitude 418781000 becomes the string "418781000" before reaching the relay.

Relay ingestion

The relay receives TelemetryFrame over the AgentGateway gRPC stream. It performs validation, copies Fields into map[string]any, assigns relay receive time, marshals the protobuf frame into Raw, and routes the resulting TelemetryEnvelope.

Although Fields is map[string]any inside the relay, its values remain strings. The conversion does not currently parse values, convert units, apply sentinel rules, or attach the complete platform identity context.

Generic outputs

File, Kafka, S3, and similar sinks receive the generic envelope. This is desirable for broad interoperability and archival because messages do not need a platform-specific converter before they can be forwarded.

Internal outputs

The registry reporter and telemetry writer are currently no-op consumers. Their routing boundaries exist, but they do not yet update registry state or persist normalized telemetry.

InfluxDB

The existing generic InfluxDB sink buffers envelopes, but `convertToInfluxPoint` discards the point it constructs and returns `nil`. It does not copy envelope fields into the point. It therefore does not implement the hot telemetry store described by the platform workstreams.

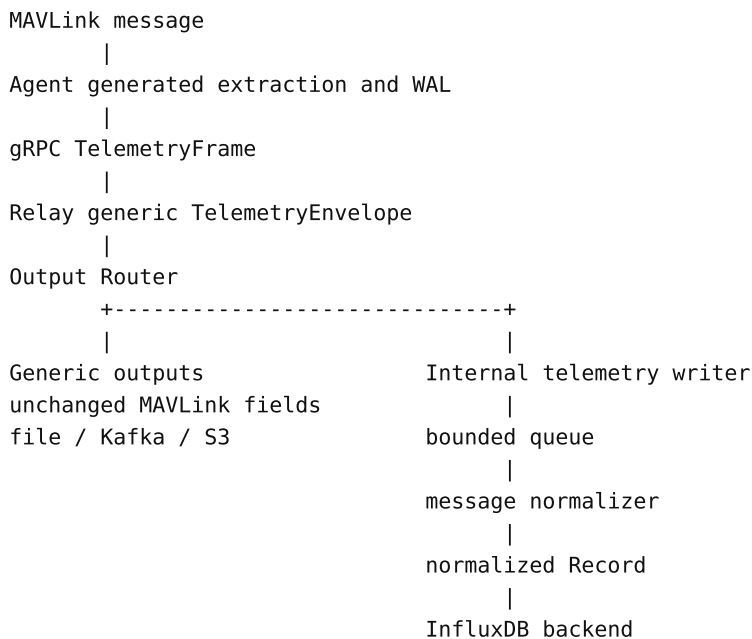
Design Goals

1. Preserve broad forwarding of common-dialect MAVLink messages.
2. Define a stable, versioned Aero Arc telemetry contract for selected messages.
3. Store numeric telemetry as numeric InfluxDB fields with documented units.
4. Keep storage-specific behavior out of the router and gRPC ingestion handler.
5. Keep the agent focused on capture, WAL durability, and delivery rather than cloud storage schemas.
6. Bound normalization and storage work so a slow backend cannot create unbounded relay work.
7. Make delivery, normalization failures, queue pressure, and backend failures observable.
8. Create a contract the API can use for latest and historical telemetry queries.

Non-Goals

- Semantically normalize every common-dialect message in the first release.
- Change generic sink payloads into the Aero Arc internal schema.
- Make the relay product API own historical telemetry queries.
- Complete replay archival, manifests, or PostGIS deconfliction in this work.
- Claim durable storage when an ACK only represents in-memory relay acceptance.

Core Architecture



The generic envelope is the broad transport contract. The normalized Record is the narrower Aero Arc product contract. These types must remain separate so adding or changing an internal schema does not break archival or operator-managed outputs.

Contracts

Generic TelemetryEnvelope

The generic envelope should preserve source data and transport metadata. It should carry at least agent identity, message ID, canonical or original message name, dialect, sequence, system ID, component ID, relay receive time, agent send time, fields, and raw payload semantics.

Generic fields may initially remain strings. Improving the protobuf to preserve native scalar types is valuable but can be a separate contract migration.

Normalized Record

Introduce a storage-neutral type owned by the internal telemetry path. A first version should contain:

```

schema_version
operator_id
aircraft_id
agent_id
relay_id
session_id
flight_id
intent_id
frame_id or an equivalent idempotency key
sequence
message_id
message_name
event_time
device_time
relay_time
timestamp_source
typed normalized fields

```

Not every identity will be available immediately. Missing identity must be explicit rather than inferred from unrelated MAVLink fields. The design must identify which system is authoritative for each ID and how the relay receives the association.

Schema versioning

Every normalized record should include a schema version. Backward-compatible field additions may remain within a version. Renames, unit changes, type changes, or semantic changes require a new version or a controlled migration.

Message Normalization

Normalization should be implemented behind the internal telemetry writer. The router should only select and deliver envelopes; it should not know field units or InfluxDB schemas.

Use a registry keyed by canonical message name. Each normalizer returns the same Record type. Separate record types for every MAVLink message are not required.

Initial message set

Start with the messages needed by Aero Arc product queries:

- Heartbeat: vehicle type, autopilot, mode, system state, and protocol version.
- GlobalPositionInt: latitude, longitude, altitude, relative altitude, velocity, and heading.
- Attitude: roll, pitch, yaw, and angular rates.

- VFR_HUD: airspeed, groundspeed, heading, throttle, altitude, and climb rate.
- SystemStatus: battery voltage, current, remaining percentage, load, communications errors, and sensor health.

The registry subscription should be reviewed separately. Registry owns liveness and placement, while geographic position and historical samples belong to telemetry. GlobalPositionInt should not be sent to registry merely because it is useful to dashboards.

Field rules

For each normalized field, document:

- Source MAVLink message and field.
- Canonical Aero Arc field name.
- Output type.
- Output unit.
- Scale or conversion formula.
- Invalid or sentinel values.
- Required versus optional behavior.
- Expected range where meaningful.

Example

GLOBAL_POSITION_INT carries latitude and longitude as degrees multiplied by 1e7, altitude in millimeters, velocities in centimeters per second, and heading in centidegrees with UINT16_MAX representing unknown.

The normalized record should use fields such as latitude_deg, longitude_deg, altitude_m, velocity_north_mps, and heading_deg. Unknown heading should be absent or null, not 655.35 degrees.

Package Boundaries

Recommended layout:

```
internal/telemetrynormalize/  
  types.go  
  registry.go  
  fields.go  
  heartbeat.go  
  global_position_int.go  
  attitude.go  
  vfr_hud.go  
  system_status.go
```

```
internal/telemetrywriter/  
  writer.go  
  store.go  
  noop.go  
  memory/  
  influx/
```

telemetrynormalize owns parsing, naming, scaling, units, and sentinel handling.

telemetrywriter owns queueing and coordination. telemetrywriter/influx owns only the mapping from normalized Record to InfluxDB points. The API reads from an API-side telemetry store implementation rather than importing MAVLink parsing logic.

InfluxDB Schema

The workstreams document proposes a stable aircraft_telemetry measurement. The initial implementation should validate that design against real query patterns before finalizing it.

Candidate tags include operator_id, aircraft_id, agent_id, relay_id, flight_id, intent_id, message_name, and schema_version. Tags should only be used for dimensions needed in filtering or grouping. High-entropy values such as frame ID, sequence, raw error text, or arbitrary field values must not become tags.

Fields should be typed normalized values such as latitude_deg, longitude_deg, altitude_m, velocity_mps, heading_deg, and battery_remaining_pct. InfluxDB does not convert numeric strings automatically, and writing a field first as a string can create later field-type conflicts.

Use trusted device event time when available and valid; otherwise use relay time. Store enough timestamp metadata to explain which source was selected.

Hot-Path And Delivery Design

The gRPC receive loop must not synchronously wait for InfluxDB. The internal telemetry writer should accept matching envelopes into a bounded queue. A fixed number of workers should normalize records and submit batches to the backend.

Required policies include queue capacity, drop or block behavior, normalization error handling, backend retry behavior, shutdown draining, and maximum write latency.

ACK semantics must be documented. An ACK after an in-memory queue accepts a frame is not a durable-storage acknowledgement. If the agent deletes WAL entries after that ACK, a relay crash may lose accepted but unwritten telemetry. A future durable relay spool or partitioned broker can provide a stronger boundary.

Observability

Add bounded-cardinality metrics for:

- Internal telemetry records accepted.
- Normalization successes and failures by canonical message name and reason category.
- Unsupported messages reaching the internal writer.
- Queue depth, queue capacity, drops, and blocking duration.
- InfluxDB batch size, latency, retries, and failures.
- Shutdown drain duration and undrained records.

Do not use agent ID, aircraft ID, flight ID, or frame ID as Prometheus labels. Those values are appropriate query dimensions in telemetry storage but create unbounded Prometheus cardinality.

Testing Strategy

Contract tests

- Canonical message-name normalization is stable.
- Each supported internal message has a registered normalizer.
- Every documented normalized field has the expected type and unit.

Golden conversion tests

For each initial message, provide representative input fields and compare the complete normalized Record. Include missing fields, malformed strings, boundary values, sentinel values, and negative values.

InfluxDB point tests

- Measurement, tags, fields, and timestamp are correct.
- Numeric fields remain numeric.
- Optional values are omitted correctly.
- High-cardinality identities are not accidentally added as inappropriate tags or metrics.

Queue and lifecycle tests

- Full-queue behavior matches policy.
- Worker failures do not terminate ingestion.
- Close drains or reports remaining work within a deadline.
- Backend errors are observable and are not silently discarded by asynchronous clients.

Implementation Phases

Phase 1: contract

1. Expand supported_messages.md into a normalization contract.
2. Define canonical message names, fields, types, units, and sentinel rules.
3. Define identity authority and timestamp policy.
4. Define ACK and durability semantics for the current milestone.

Phase 2: normalizer

1. Create the storage-neutral Record type.
2. Create robust typed field-parsing helpers.
3. Implement normalizers for the initial message set.
4. Add golden tests and normalization metrics.

Phase 3: internal telemetry writer

1. Replace the no-op telemetry writer with a bounded asynchronous writer.
2. Add a memory backend for tests and local demos.
3. Define queue, retry, and shutdown policies.

Phase 4: InfluxDB backend

1. Implement normalized Record to InfluxDB point conversion.
2. Batch writes and consume asynchronous client errors, or use a blocking batch API in workers.
3. Add integration tests and an operational dashboard.
4. Clarify or retire the existing generic InfluxDB sink to avoid confusing it with the official normalized telemetry backend.

Phase 5: API reader

1. Implement the API telemetry.Store InfluxDB reader.
2. Support latest sample, flight track, and time-window queries.
3. Keep memory storage for demos and tests.

Phase 6: typed generic transport

As a separate compatibility project, evolve TelemetryFrame.Fields from map<string, string> to typed protobuf values. Extend the existing generation-time reflection tool so all common-dialect messages preserve native scalar and array types. This improves generic sinks and reduces parsing overhead, but it is not required to complete the first normalized InfluxDB slice.

Relationship To Platform Workstreams

This plan directly advances Workstream 2, InfluxDB Telemetry. It finishes the missing relay point conversion by first establishing the stable schema that point conversion must implement. It also advances the cross-cutting Identity and Contracts workstream by forcing identity authority, idempotency, timestamp, and schema-version decisions.

It supports Workstream 1 by keeping registry normalization separate from hot telemetry and by clarifying that registry owns reachability and placement rather than historical aircraft samples.

It prepares Workstreams 3 and 4 because normalized telemetry, stable flight identity, event timestamps, and idempotency are prerequisites for querying completed flights and producing deterministic replay artifacts. It does not itself implement archival jobs, S3 replay objects, manifests, or presigned URLs.

It does not directly advance PostGIS deconfliction, except that stable aircraft, flight, intent, position, and timestamp contracts will later make spatial/temporal integration safer.

Definition Of Done For The First Vertical Slice

- The five initial messages have documented and tested normalization contracts.
- The relay produces typed normalized records without changing generic sink envelopes.
- Internal telemetry delivery is bounded and observable.
- InfluxDB stores usable numeric fields under a stable measurement schema.
- Backend write failures are surfaced rather than silently discarded.
- The API can query latest and historical telemetry through its telemetry.Store interface.
- Identity, timestamp, schema version, and current durability guarantees are documented.

Decision Summary

- Generic MAVLink forwarding remains broad and unchanged.
- Aero Arc normalizes only the messages it treats as internal product telemetry.
- Normalization occurs behind the relay telemetry writer, not in API handlers or the router.
- InfluxDB consumes normalized records, not raw string-valued envelopes.
- The agent remains responsible for capture, WAL, retry, and delivery.
- Typed all-message protobuf transport is valuable follow-up work, not a blocker for the first slice.

This is the correct continuation of the platform workstreams: it hardens the relay-to-telemetry-store seam and turns the existing routing scaffold into the first half of a production-style Aero Arc telemetry path.